A Pen and a Paper in 3D Space

Computer Graphics & Animation Project Report

1. Project Requirements

The primary goal of this project is to implement an interactive 3D workspace scene using **Three.js**. The scene depicts a notebook paper sheet resting on a wooden desk with an active pen drawing on it. The implementation successfully incorporates:

- **Custom GLSL Shaders**: Custom vertex/fragment shaders for marble and holographic pen finishes.
- **Lighting**: SpotLight desk lamp with soft shadows, localized pen tip glow, and ambient room fill.
- **Perspective Projection**: Bounded perspective viewport camera matrix calculations.
- **Textures**: Procedural desk wood grain, lined notebook paper, paper fibers, and pencil cedar wood.
- **Animations**: Spline cursive writing path, automatic spirograph tracing, and kinetic pen tilting/lifting.
- **Interactivity**: Keyboard spherical camera orbiting, mouse orbital navigation, draw mode, and pen switching.

2. Software Platform

The project is built entirely on web standards running natively in any modern web browser:

- **HTML5 / CSS3**: Structured elements and glassmorphic UI overlay panels.
- **JavaScript (ES6 Modules)**: Modular application logic.
- **Three.js (r147) & OrbitControls**: WebGL 3D rendering and camera controllers.
- **HTML5 Canvas API**: Real-time procedural texture mapping.

3. Project Features

3.1 Models and Texture Mapping

All 3D objects and meshes are built using Three.js geometries, combined with dynamic custom texture maps:

- **The Desk**: Mapped with a procedural dark walnut wood texture generated using high-frequency noise fibers and wood knot wave calculations on startup.
- **The Paper**: A box with a dynamic canvas writing texture displaying rule lines, margins, and pulpy fibers.
- **The Pens & Glass Holder**: Swappable pen geometries (marble Fountain Pen, holographic Modern Stylus, cedar Wooden Pencil) and a translucent glass cup where unused pens dynamically rest.

3.2 Key Interaction

Allows camera orbiting around the paper using spherical coordinates:

- `W` / `ArrowUp` and `S` / `ArrowDown` adjust the polar angle (ϕ).
- `A` / `ArrowLeft` and `D` / `ArrowRight` adjust the azimuthal angle (θ).
- `Space` smoothly resets the camera back to the original isometric viewport angle.

3.3 Mouse Interaction

Context-aware: OrbitControls are used for viewing, while toggling "Enable Mouse Drawing" projects a ray onto the paper. Dragging paints ink strokes onto the canvas, and the 3D pen follows the cursor.

3.4 Animation

The auto-write engine moves the pen along a cursive path ("Three.js" and a spirograph). The pen tilts dynamically based on its motion velocity vector and lifts/drops realistically during transitions.

3.5 Lighting

Uses a warm spotlight lamp ($0xfff1e0$, intensity 2.2) with soft shadow-mapping (`THREE.PCFSoftShadowMap`), ambient room fill, and a localized point light at the stylus tip for a glowing neon bloom.

3.6 Custom Shaders

Implemented via `THREE.ShaderMaterial`:

- **Marble Shader (Fountain Pen)**: Fragment shader uses Fractional Brownian Motion (fBm) noise to blend forest green and cream-gold veins under a Blinn-Phong specular reflection model.
- **Holographic Shader (Stylus)**: Fragment shader computes the Fresnel viewing angle to shift the body color toward a time-animated rainbow gradient, creating a thin-film iridescent metal look.

3.7 Project Feature Status Table

#	Feature Name	Classification	Description
1	Custom Shaders	**Implemented**	Custom GLSL fBm marble noise and holographic Fresnel shaders.
2	Lighting & Shadows	**Implemented**	Soft shadows via SpotLight, ambient fill, and pen tip light.
3	Perspective Projection	**Implemented**	Perspective camera calibration and Target orbiting constraints.
4	Procedural Textures	**Implemented**	Desk wood grain, lined notebook page, and graphite lead.
5	Spline Animation	**Implemented**	Continuous cursive script and spirograph curves drawing.
6	Keyboard Camera Orbit	**Implemented**	Spherical orbiting using WASD/Arrows and Space reset.
7	Mouse Interaction	**Implemented**	Raycasting coordinate mapping to paint custom ink drawings.
8	Model Swapping UI	**Implemented**	Swappable Fountain Pen, Modern Stylus, and Wooden Pencil.
9	Interactive Pen Holder	**Implemented**	Glass holder that automatically updates containing unused pens.

Table 01: Project Feature Table

4. Snapshots

Figure 01: High-fidelity rendered 3D scene displaying writing path, desk lamp shadows, and marble shader.

5. Contribution

The project development tasks were distributed among the group members as follows:

Student Name	Student ID	Contribution & Tasks Performed
Ferdaous Wahid	2022100010077	Shader implementations (vertex and fragment), mathematical spirograph equations, and keyboard orbiting camera physics.

Student Name	Student ID	Contribution & Tasks Performed
Abdullah Al Noman	2022200010031	Geometry modeling of the pens, desk lamp, glass holder, and UI overlay panel styling sheets.
Javer Hossain	2022100010086	Procedural canvas texture generators, raycasting drawing coordinates mapping, and spline path transitions.

Table 02: Contribution Table